

SYSTEM DESIGN REPORT

AetherGrid AI

Autonomous Multi-Agent Logistics and
Emergency Dispatch Simulation

Document no.	AEG-RPT-001
Status	Draft for design review
Prepared by	Design Engineering
Review panel	To be assigned at design review

Role	Name	Roll Number:
Author 1	Sparsh Verma	1024240011
Author 2	Vishal Singla	1024240009

Revision history. Rev 0.1.0: initial draft issued for design review. All quantitative figures are design projections; no empirical results are reported (see §1.4).

Contents

1 Introduction 4

1.1 Purpose and scope 4

1.2 Design tenets 4

1.3 Definitions and abbreviations 4

1.4 Statement on data provenance 4

1.5 Document family and evolution 5

2 Problem Formulation 5

3 System Architecture 6

3.1 Topology 6

3.2 Component responsibilities 6

3.3 Interfaces and determinism 6

3.4 Dependency policy 7

4 Simulation Model 7

4.1 Environment coefficients 7

4.2 Agent dynamics 7

4.3 Incident model 8

5 Algorithmic Framework 8

5.1 Edge cost model 8

5.2 Pathfinding 8

5.3 Assignment model 9

5.4 Re-positioning policy (RL) 9

5.5 System composition 10

5.6 Compute budget 10

6 Mission-Control Interface 10

7 Verification and Validation 11

7.1 Pre-registered hypotheses 11

7.2 Benchmark protocol 11

7.3	Step-boundary invariants	12
7.4	Test plan	12
8	Implementation Plan	12
8.1	Phases and exit criteria	12
8.2	Repository layout	13
9	Risks and Open Questions	13
10	Conclusion	13
A	Notation	14
B	Parameter Defaults	14
C	Worked Example: One Batch Window	15

List of Figures

1	System topology. Dispatch strategies are injected as callbacks; the engine runs identically headless (CI) or attached to the dashboard.	6
2	Agent finite-state machine. Transitions are asserted at step boundaries (I2, I6). .	7
3	Diurnal arrival-rate schedule $\lambda(t)$ (design parameter).	8
4	Worked routing example: the wall forces a detour; hazards raise cost without blocking. Shaded: closed set; dashed: open frontier.	9
5	Per-window control loop. Every stage reads from the cost field, and the RL layer feeds back over the idle remainder of the assignment.	10
6	Design projections used only to size the Phase 4 benchmark (see the statement on data provenance, §1.4).	11
7	Indicative schedule (weeks).	13

List of Tables

1	Abbreviations used in this document.	4
2	Document family. Child documents inherit the definitions, assumptions and invariants of this baseline. Later revisions are recorded on the cover page. . . .	5
3	Component responsibilities and repository mapping.	6
4	Travel-time multipliers. Defaults: $\alpha = 0.85$, $\beta = 45$ s.	7

5	Incident taxonomy and SLA contract. Breaches are logged, never discarded. . .	8
6	MDP specification of the re-positioning layer.	9
7	Per-window compute budget. $\Delta = 5\text{ s}$ is an operational choice (batching gain vs. response latency), not a compute limit.	10
8	Phase plan with exit criteria.	12
9	Risk register (L = likelihood, I = impact: L/M/H).	13
10	Principal notation.	14
11	Tunable defaults (all overridable per scenario).	14

1 Introduction

1.1 Purpose and scope

This report sets out the design of **AetherGrid AI**, a discrete-event simulation engine and mission-control platform for autonomous fleet routing, emergency triage dispatch, dynamic pathfinding, and reinforcement-learning (RL) fleet re-positioning. Section 2 formulates the problem, §3 describes the architecture, §4 the simulation model, §5 the algorithmic framework, §6 the mission-control interface, §7 verification and validation, and §8 the implementation plan.

1.2 Design tenets

Four principles shape the design:

1. **Determinism.** A fixed seed produces a byte-identical metric log, verified by SHA-256, so every experiment in §7 can be repeated exactly.
2. **Pluggability.** Dispatch and routing strategies are runtime callbacks. Any of them can be swapped without touching the engine.
3. **Lean runtime.** Python standard library plus NumPy, in an isolated `venv`. No frameworks.
4. **Unit consistency.** Every cost in the system is expressed in seconds, which lets distance, congestion, weather and deadlines be compared directly.

1.3 Definitions and abbreviations

ETA	Estimated time of arrival (shortest-path cost on the priced graph)
SLA	Service-level agreement; maximum admissible response time
VRPTW	Vehicle routing problem with time windows
FSM	Finite-state machine
EWMA	Exponentially weighted moving average
RL	Reinforcement learning

Table 1: Abbreviations used in this document.

1.4 Statement on data provenance

Data provenance

AetherGrid AI has not been implemented yet, so this report contains no measurements. Figures labelled *projected* are design projections, used only to size the Phase 4 benchmark (§7.2). The empirical commitments of the project are the three pre-registered hypotheses in §7.1, each written down with its test protocol and falsification condition before any data exist. If the data falsify a hypothesis, that counts as a valid result.

1.5 Document family and evolution

This report is revision 0.1.0 of the system-level design baseline, not the final word. Detailed design, implementation and validation will land as separate controlled documents, and measured results will replace every figure currently marked *projected*.

ID	DOCUMENT	STAGE	STATUS
AEG-RPT-001	System Design Report (this document)	Design review	Rev 0.1.0 draft
AEG-DDS-001	Detailed design: core simulation	Phase A	Planned
AEG-DDS-002	Detailed design: AI suite (routing, dispatch, RL)	Phase B	Planned
AEG-DDS-003	Detailed design: mission control	Phase C	Planned
AEG-VER-001	Verification plan and results (Hypotheses 7.1–7.3)	Phase D	Planned

Table 2: Document family. Child documents inherit the definitions, assumptions and invariants of this baseline. Later revisions are recorded on the cover page.

2 Problem Formulation

Definition 2.1 (Dynamic transportation network). At time t the city is a directed graph $G_t = (V, E_t)$, $E_t \subseteq V \times V$. Each edge e carries length ℓ_e , terrain class $\tau_e \geq 1$, congestion density $\rho_e(t) \in [0, 1]$ (EWMA of occupancy), a weather multiplier $\omega(t) \geq 1$, and a hazard indicator. A *closure* removes e from E_t ; a *hazard* retains e at increased cost.

Definition 2.2 (Fleet). A set $\mathcal{A}$ of heterogeneous agents (ambulance, fire engine, police, drone, delivery van). Agent a has position $\text{pos}_a(t)$, battery $B_a(t) \in [0, 1]$, capacity, and status in {IDLE, ENROUTE, SERVICING, RECHARGING}.

Definition 2.3 (Demand process). Calls arrive as a marked Poisson process $N(t) \sim \text{Poisson}(\lambda t)$ with diurnal rate $\lambda(t)$; call j is the tuple $(\text{loc}_j, w_j, D_j, \text{category})$ with priority weight w_j , arrival deadline D_j , and log-normal service time.

Definition 2.4 (Dispatch policy). Every batch window Δ , a policy π maps the buffered calls $B(t)$, the idle agents $I(t)$ and the network state G_t to an assignment $x \in \{0, 1\}^{I \times B}$. Assigned agents follow least-cost paths; unassigned calls stay in $B(t)$; idle agents may be relocated by a re-positioning policy between windows.

Objective. Minimise expected weighted response $\mathbb{E}[\sum_j w_j T_j]$ subject to SLA fulfilment $P(T_j \leq D_j)$ at the required levels, with online, non-clairvoyant information.

Proposition 2.1 (Complexity). For m idle agents and n buffered calls, exhaustive assignment evaluates m^n candidates ($20^{12} \approx 4.09 \times 10^{15}$ at fleet scale). The Hungarian method recovers the exact optimum in $O(n^3)$ [2]. When each cost entry is a shortest path on G_t , the per-batch work is dominated by $|I| \cdot |B|$ shortest-path queries. Multi-stop logistics adds per-vehicle stop sequencing (VRPTW), which is NP-hard [3].

Exhaustive search is therefore out of the question, and the design uses layered structure instead: a pricing layer (§5.1), an exact assignment layer (§5.3), and a learned pre-positioning layer

(§5.4).

3 System Architecture

3.1 Topology

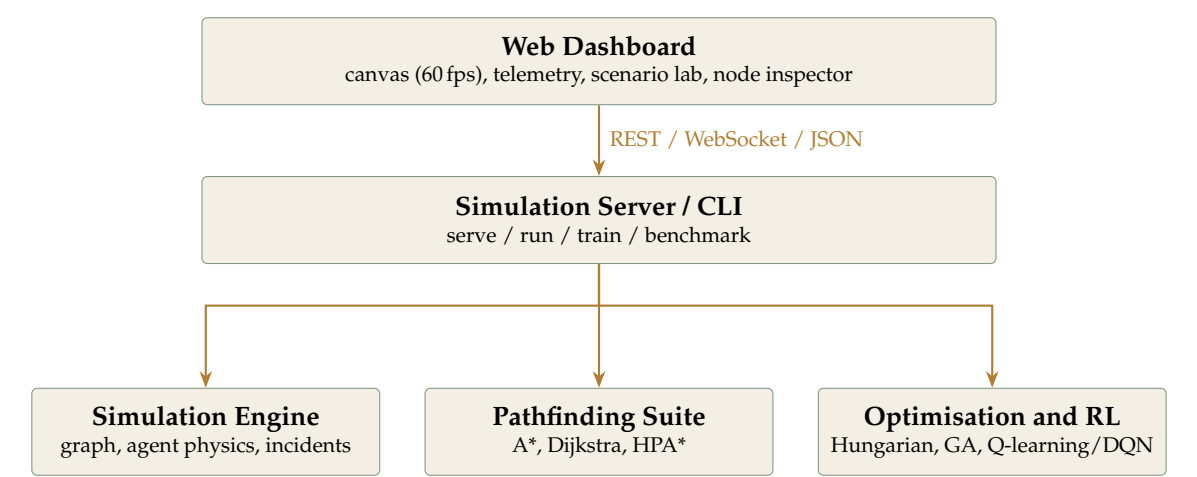


Figure 1: System topology. Dispatch strategies are injected as callbacks; the engine runs identically headless (CI) or attached to the dashboard.

3.2 Component responsibilities

Module	Responsibility	Path
Simulation Engine	graph, agent FSM and physics, Poisson generator, event loop	core/
Pathfinding Suite	dynamic A*, Dijkstra ETA matrices, hierarchical HPA*	ai/pathfinding.py
Dispatchers	greedy baseline, Hungarian matcher, genetic VRPTW	ai/dispatchers.py
RL Core	Q-learning → DQN idle re-positioning	ai/rl_agent.py
Mission Control	HTTP/WS server, canvas visualiser, telemetry, scenario lab	web/

Table 3: Component responsibilities and repository mapping.

3.3 Interfaces and determinism

The dashboard receives state diffs at 10 Hz and interpolates them to 60 fps. Commands flow the other way: inject hazard, close edge, set weather, switch strategy. All randomness passes through a single seeded generator (Assumption 3.1).

Assumption 3.1 (Determinism). With a fixed seed, the sequence of arrivals, service times and tie-breaks is reproduced exactly; metric logs are byte-identical across runs.

3.4 Dependency policy

The runtime is stdlib plus NumPy. We hand-roll the web layer over HTTP/WebSocket rather than adopting a framework, and the engine is importable, so tests, notebooks and the server all embed the same `SimulationEngine`.

4 Simulation Model

4.1 Environment coefficients

TERRAIN	τ_e	WEATHER	$\omega(t)$
Highway	1.00	Clear	1.00
Arterial	1.25	Rain	1.22
Urban	1.45	Fog	1.45
Alley	1.90	Snow	1.85
Offroad	3.10		

Table 4: Travel-time multipliers. Defaults: $\alpha = 0.85$, $\beta = 45$ s.

Closure vs. hazard (normative)

A **closed** edge is removed from E_t and must force re-routing; a **hazard** remains traversable at penalty β . Implementations must not conflate the two: the distinction matters for admissibility (Property 5.1) and for invariant I5 (§7.3).

4.2 Agent dynamics

Battery evolves as

$$B_a(t + \Delta t) = B_a(t) - \Delta t (p_0 + p_1 v \text{ load}) / E_{\text{cap}}, \quad (1)$$

with logistic charging. Emergency agents apply siren mode (congestion weight $\times 0.35$) and priority escalation ($w_j += 0.2$ per minute of SLA excess); logistics agents run capacity-constrained multi-drop routes.

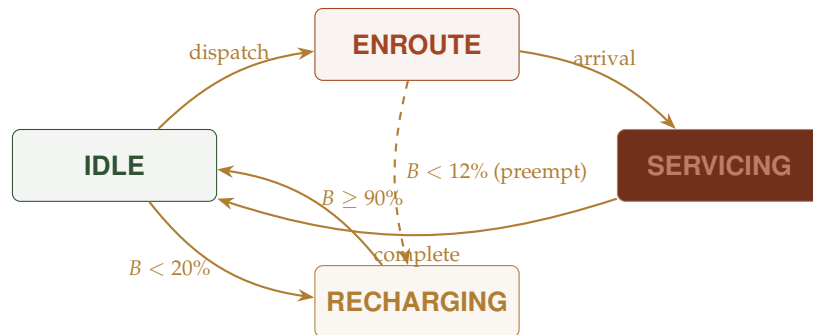


Figure 2: Agent finite-state machine. Transitions are asserted at step boundaries (I2, I6).

4.3 Incident model

Inter-arrival times are $\text{Exp}(\lambda)$; the diurnal schedule (Figure 3) applies rush multipliers of $2.4\lambda_0$.

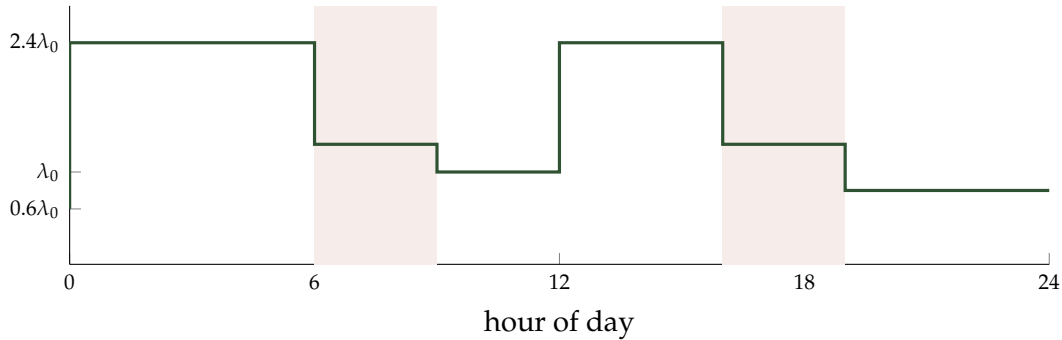


Figure 3: Diurnal arrival-rate schedule $\lambda(t)$ (design parameter).

CATEGORY	w_j	DEADLINE	SERVICE	NOTES
Emergency medical	1.00	≤ 8 min	6–12 min	escalation on breach
Structure fire	0.90	≤ 10 min	15–25 min	fire engine required
Hazmat	0.95	≤ 12 min	20 min	corridor exclusion
Package delivery	0.40	[earliest, due]	2–4 min	soft window

Table 5: Incident taxonomy and SLA contract. Breaches are logged, never discarded.

Queue diagnostics use Little’s law $L = \lambda W$ [6], reported per zone.

5 Algorithmic Framework

5.1 Edge cost model

Definition 5.1 (Edge cost).

$$c(e, t) = \ell_e \tau_e (1 + \alpha \rho_e(t)) \omega(t) + \beta \mathbf{1}[\text{hazard}_e], \quad (2)$$

where ℓ_e is free-flow traversal time. Terrain, congestion and weather enter multiplicatively; hazards additively; closures remove the edge (§4.1).

Everything downstream sees only $c(e, t)$. No component inspects raw map state.

5.2 Pathfinding

Live dispatch runs on A* with the heuristic below. The Hungarian matcher reads cached Dijkstra distance matrices, recomputed only when the topology changes. Long-haul queries use HPA* with precomputed boundary graphs over 8×8 chunks [4].

Property 5.1 (Admissibility). *With $h(n) = \tau_{\min} \cdot d(n, \text{goal})$ and $\tau_e, \omega \geq 1$, h never overestimates*

residual cost, so A^* returns optimal paths on the priced graph [1]. Hazmat injections raise β without removing edges, preserving this property.

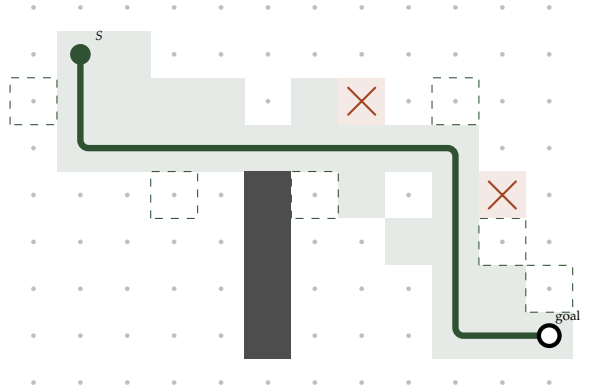


Figure 4: Worked routing example: the wall forces a detour; hazards raise cost without blocking. Shaded: closed set; dashed: open frontier.

5.3 Assignment model

Definition 5.2 (Batch assignment). Every $\Delta = 5$ s, solve

$$\min_x \sum_i \sum_j c_{ij} x_{ij} \quad \text{s.t.} \quad \sum_j x_{ij} \leq 1 \quad \forall i, \quad \sum_i x_{ij} \leq 1 \quad \forall j, \quad x \in \{0, 1\}, \quad (3)$$

$$c_{ij} = \text{ETA}_{ij} + \mu \max(0, t + \text{ETA}_{ij} - D_j), \quad \text{ETA}_{ij} = \text{dist}_{c(t)}(\text{pos}_i, \text{call}_j). \quad (4)$$

The Hungarian method [2] gives the exact optimum. The matcher only ever sees the cost matrix from Definition 5.1, never the graph itself, which is why strategies can be hot-swapped.

Property 5.2 (Always feasible). *Because deadline violations enter as a finite penalty μ rather than hard constraints, (3) stays feasible for every buffer/fleet configuration, including demand bursts. Rectangular instances ($|B| \neq |I|$) are handled by padding. The $\leq$ constraints are deliberately loose: leftover calls re-buffer, and leftover agents feed the policy of §5.4.*

5.4 Re-positioning policy (RL)

State	(sector, time bin, fleet occupancy vector)
Action	stay; relocate to adjacent sector
Reward	$r = -\sum_j w_j T_j - \kappa \Delta E + b \mathbf{1}[\text{SLA met}]$
Horizon	$\gamma = 0.97$; decisions every 30 s, idle agents only

Table 6: MDP specification of the re-positioning layer.

Definition 5.3 (Update rule). Q-learning [5], one-step, off-policy:

$$Q(s, a) \leftarrow Q(s, a) + \eta [r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (5)$$

with learning rate $\eta \in (0, 1]$ ($\eta = 0.1$; ε -greedy exploration, $\varepsilon : 1.0 \rightarrow 0.05$). The tabular baseline is succeeded by DQN (replay buffer 50 000, target-network sync every 1 000 steps). We write η for the learning rate because α is already taken in (2).

Assumption 5.1 (Layer separation). The re-positioning policy touches only idle agents, only between batch windows, and at sector granularity. It never overrides an active dispatch. With that boundary in place, the assignment layer of §5.3 stays exact and unmodified.

5.5 System composition

The layers run as one loop per batch window (Figure 5). The cost field (Definition 5.1) is maintained continuously; the ETA matrix is derived from it; the assignment consumes the matrix; dispatch executes; and the RL layer relocates the idle remainder before the next window.

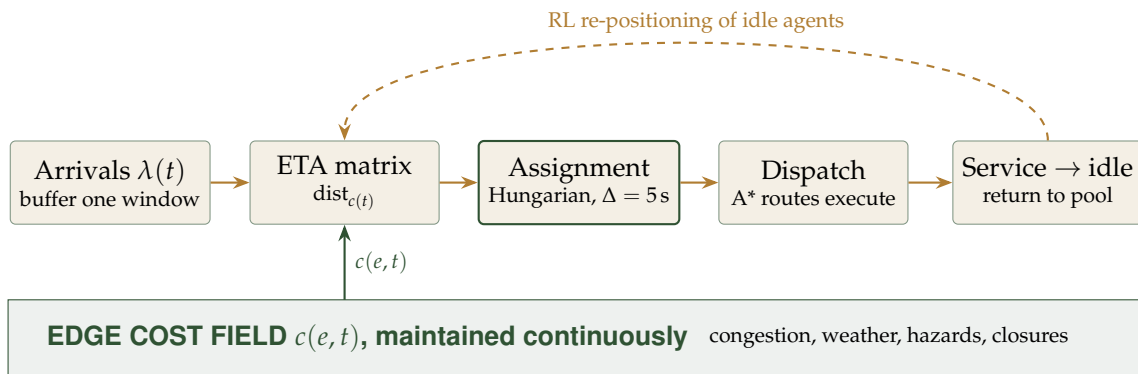


Figure 5: Per-window control loop. Every stage reads from the cost field, and the RL layer feeds back over the idle remainder of the assignment.

5.6 Compute budget

STAGE	COMPLEXITY	NOTES
Cost-field update	$O(E)$	EWMA per edge per tick
ETA queries	cached shortest paths	~1–2 ms each on reference grid
Assignment	$O(n^3)$	< 2 ms for $n \leq 16$
Re-positioning	$O(1)$ per idle agent	table lookup / small forward pass

Table 7: Per-window compute budget. $\Delta = 5$ s is an operational choice (batching gain vs. response latency), not a compute limit.

6 Mission-Control Interface

The following is to be implemented in Phase 3:

- Canvas visualisation at 60 fps interpolated from 10 Hz WebSocket diffs.
- Telemetry: response times, SLA fulfilment, battery utilisation, queue lengths.

- Per-node inspector exposing $c(e, t), \rho_e, \omega$.
- Scenario lab: inject hazmat, induce rush-hour demand, close edges.
- Strategy selector for live hot-swap between dispatchers.
- Benchmark mode: side-by-side execution on identical seeded call streams, JSON export.

7 Verification and Validation

7.1 Pre-registered hypotheses

We pre-register three hypotheses before any data exist. Directionally, we expect $T_{\text{greedy}} > T_{\text{hungarian}} > T_{\text{RL}}$ on mean response time.

Hypothesis 7.1 (Matching beats greedy dispatch). *Claim.* Under bursty demand, batch-optimal assignment yields lower mean response time and higher SLA fulfilment than myopic nearest-unit dispatch.

Test. Seeded A/B runs with identical call streams, ≥ 5 seeds.

Falsified if no statistically significant separation is observed.

Hypothesis 7.2 (Pre-positioning compounds). *Claim.* RL pre-positioning improves on assignment alone, because ETAs shrink before matching.

Test. Hungarian vs. RL+Hungarian under identical seeds.

Falsified if the marginal gain ≈ 0 or the energy cost exceeds it.

Hypothesis 7.3 (Robustness under disruption). *Claim.* The cost model absorbs hazards and closures: affected agents re-route within one tick and invariants I1–I6 hold.

Test. Scenario injections mid-run with trace replay.

Falsified if re-route lags or any invariant is violated.

7.2 Benchmark protocol

Reference scenario: 24×24 grid, 9 vehicles, diurnal λ (Figure 3), 12 min episodes $\times$ 5 seeds per strategy; identical call streams across strategies; JSON reports; regressions gate merges at $\pm 5\%$.

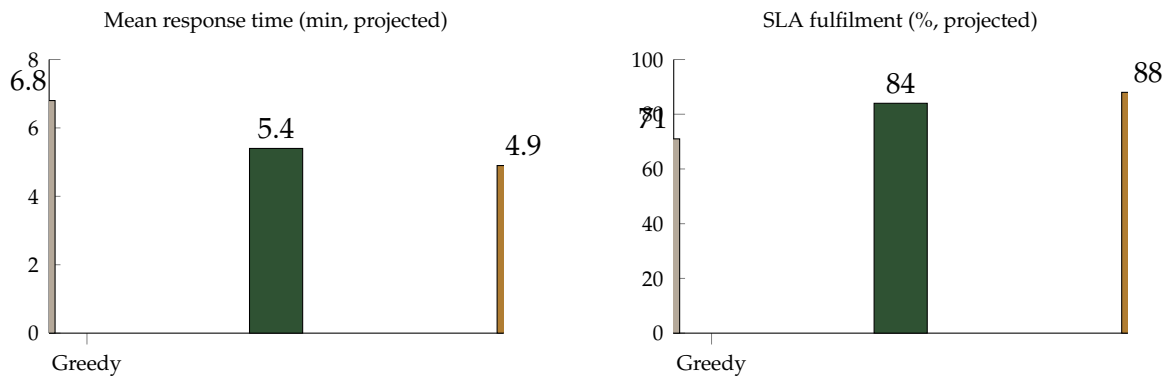


Figure 6: Design projections used only to size the Phase 4 benchmark (see the statement on data provenance, §1.4).

7.3 Step-boundary invariants

Property 7.1 (Runtime invariants). *Asserted at every simulation tick:*

- I1. No agent occupies a closed edge.*
- I2. $B_a \in [0, 1]$; charging transitions are strictly monotonic.*
- I3. Every completed incident carries a full path trace; SLA breaches are logged, never discarded.*
- I4. Identical seed $\Rightarrow$ byte-identical metric log (SHA-256).*
- I5. Hungarian batch cost $\leq$ greedy batch cost on the same window (cross-validated against brute force for $n \leq 8$).*
- I6. Fleet conservation: $|\text{IDLE}| + |\text{ENROUTE}| + |\text{SERVICING}| + |\text{RECHARGING}| = |\mathcal{A}|$.*

7.4 Test plan

```
# automated suite
./venv/bin/python3 -m unittest discover -s tests
# algorithm benchmark across strategies
./venv/bin/python3 -m aethergrid.cli benchmark --duration 100
# manual verification (mission control)
./venv/bin/python3 -m aethergrid.cli serve --port 8080
```

Manual checklist: zoom/pan/inspect any node; hot-swap dispatcher mid-run; inject a closure and verify re-route within one tick. Coverage targets: $\geq 90\%$ on `core/`, $\geq 85\%$ on `ai/`.

8 Implementation Plan

8.1 Phases and exit criteria

Phase	Scope	Exit criterion
A	Core: grid, agents, incidents, engine	seed-stable headless run
B	AI suite: A*/Dijkstra/HPA*, Hungarian, GA, RL	property tests vs. brute force
C	Mission control web layer	live strategy hot-swap
D	Verification: benchmarks, tests, CI	Hypotheses 7.1–7.3 each confirmed or falsified

Table 8: Phase plan with exit criteria.

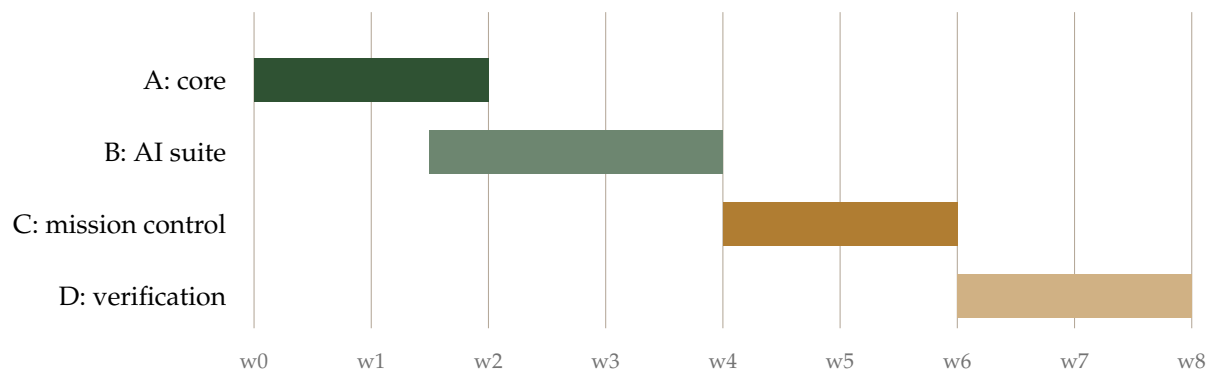


Figure 7: Indicative schedule (weeks).

8.2 Repository layout

```
aethergrid/  
|-- core/      grid.py  agents.py  incidents.py  engine.py  
|-- ai/        pathfinding.py  dispatchers.py  rl_agent.py  
|-- web/       server.py  index.html  app.js  styles.css  
`-- cli.py  
tests/        test_simulation.py  test_pathfinding.py  test_dispatch.py
```

9 Risks and Open Questions

RISK	L	I	MITIGATION
RL training instability	M	M	tabular baseline first; seeded replays; fallback to static posting
ETA cache staleness	M	H	recompute on topology events; staleness bound asserted
Benchmark variance masks effects	M	M	≥ 5 seeds; identical streams; ±5% merge gate
Web-layer scope creep	H	L	spec frozen at §6; deferred to Phase 3 exit
HPA* precompute cost	L	M	granularity sweep scheduled in Phase B

Table 9: Risk register (L = likelihood, I = impact: L/M/H).

Open questions for the panel: where the knee of batch window Δ sits, what HPA* chunk granularity to use, and whether the RL reward needs fairness terms.

10 Conclusion

The design splits dynamic dispatch into three layers that can each be verified on their own: a cost field maintained continuously, an exact batch assignment over that field, and a constrained re-positioning policy for the idle remainder. Because the whole system is deterministic by construction, every claim

made here can be tested later. If this report is approved, we begin Phase A.

References

- [1] P. E. Hart, N. J. Nilsson, B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [2] H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [3] M. M. Solomon. On the worst-case performance of some algorithms for the vehicle routing problem with time windows. *Transportation Science*, 21(1):16–29, 1987.
- [4] A. Botea, M. Müller, J. Schaeffer. Near-optimal search with continuous abstractions. *Journal of Artificial Intelligence Research*, 21:285–313, 2004.
- [5] C. J. C. H. Watkins, P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [6] J. D. C. Little. A proof for the queuing formula $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961.

A Notation

ℓ_e	free-flow traversal time	η	RL learning rate
τ_e, ω	terrain / weather multipliers	γ	discount factor (0.97)
$\rho_e(t)$	congestion density (EWMA)	μ	deadline-penalty weight
α	congestion sensitivity (0.85)	Δ	batch window (5 s)
β	hazard penalty (45 s)	D_j	arrival deadline of call j

Table 10: Principal notation.

B Parameter Defaults

PARAMETER	DEFAULT	LOCATION
α (congestion sensitivity)	0.85	cost model
β (hazard penalty)	45 s	cost model
Δ (batch window)	5 s	assignment
μ (deadline penalty)	2.0	assignment
γ (discount)	0.97	RL
η (learning rate)	0.1	RL
ε decay	1.0 $\rightarrow$ 0.05 over 200k steps	RL
Rush multiplier	$2.4\lambda_0$	demand

Table 11: Tunable defaults (all overridable per scenario).

C Worked Example: One Batch Window

Three idle agents V_1, V_2, V_3 ; buffered calls M (medical, $D_M = t + 5$ min) and F (fire, $D_F = t + 10$ min); $\mu = 2$. ETAs (min): $M : (4.2, 4.9, 6.5)$, $F : (4.6, 8.0, 7.2)$. Deadline pricing raises $c_{3M} = 6.5 + 2(1.5) = 9.5$; all other entries equal their ETAs. Greedy dispatch (calls in arrival order) selects $M \rightarrow V_1, F \rightarrow V_3$: total 11.4 min. The Hungarian optimum is $M \rightarrow V_2, F \rightarrow V_1$: total 9.5 min ($\approx 17\%$ better), leaving V_3 idle. The RL layer compares $Q(\text{stay}) = -12.4$ against $Q(\text{north}) = -8.1$, orders V_3 north, and the TD update with $\eta = 0.1, r = -6.0, \max_{s'} Q(s') = -7.5$ gives $Q \leftarrow -8.1 + 0.1(-6.0 + 0.97(-7.5) + 8.1) = -8.62$.